Republic of the Philippines
POLYTECHNIC UNIVERSITY OF THE PHILIPPINES
SAN PEDRO CAMPUS
PUP, Main Road, Barangay United Bayanihan, City of San Pedro, Laguna

Pawpaw Database Roles and Privileges

Activity 4 Progress Documentation

1. Introduction

This document presents the progress of Activity 4: Database Users, Roles, Privileges, and Access Control for the Pawpaw Clinic project.

The activity focuses on creating database roles, assigning appropriate privileges, and testing access restrictions using PostgreSQL. The goal is to apply the principle of least privilege, where each database user receives only the permissions needed for their assigned responsibilities.

2. Project Overview

The project used for this activity is Pawpaw Clinic, which has a PostgreSQL database named group3_pawpaw.

The database contains the following tables:

- appointment
- appointment_service
- audit_log
- auth_session
- medical_record
- notification
- owner
- password_reset_code
- pet
- prescription
- service
- user_account
- vaccination
- veterinarian

The database was initialized using the project's migration and seed scripts and populated with sample records for testing.

3. Database Inspection

Before creating the roles, the database structure and existing records were inspected.

The database contains:

- 14 tables
- 12 sequences
- 1 schema named public

The following row counts were recorded:

Table	Number of Records
appointment	3
appointment_service	9
audit_log	40
auth_session	0
medical_record	1
notification	26
owner	3
password_reset_code	0
pet	2
prescription	0
service	10
user_account	7
vaccination	0
veterinarian	1

The inspection helped identify the existing database objects before assigning permissions.

4. Role Creation and Configuration

Two group roles were created based on the database access requirements of the Pawpaw Clinic application.

4.1 Group Roles

Group Role	Purpose
app_admin_grp	For database administration tasks
app_runtime_grp	For normal application operations

Both group roles were created with the NOLOGIN attribute. This means they cannot be used directly to connect to the database.

4.2 Login Roles

Login Role	Assigned Group Role
pawpaw_dba	app_admin_grp
pawpaw_app	app_runtime_grp

- The login roles were created with the LOGIN attribute and assigned to their corresponding group roles.
- Passwords were configured locally and were not included in the repository.

5. Privilege Assignment

Privileges were assigned according to each role's responsibility.

5.1 Database and Schema Privileges

Both group roles were granted CONNECT permission on group3_pawpaw. The app_runtime_grp and app_admin_grp roles were also granted USAGE permission on the public schema.

To restrict database access, the CONNECT privilege was revoked from PUBLIC on group3_pawpaw. Explicit CONNECT privileges were retained for the two intended group roles.

The CREATE privilege was also restricted on the public schema. Only the administrative group (app_admin_grp) was granted CREATE permission, while the runtime group (app_runtime_grp) received USAGE permission only.

5.2 Administrative Role

The app_admin_grp role was granted ALL PRIVILEGES on all existing tables and sequences in the public schema.

It was also granted USAGE and CREATE permissions on the public schema to allow administrative database operations.

The pawpaw_dba login role was assigned as a member of app_admin_grp to inherit its privileges.

5.3 Runtime Role

The app_runtime_grp role was granted:

- SELECT
- INSERT
- UPDATE
- DELETE

It was also granted USAGE and SELECT privileges on all sequences in the public schema.
The pawpaw_app login role was assigned as a member of app_runtime_grp to inherit its privileges.

The DELETE privilege was included in the runtime role's permissions, allowing the application account to perform deletion operations when needed.

5.4 Read-Only Role

No separate read-only group role was created for the Pawpaw Clinic database.
The database access configuration only includes two group roles:

- app_admin_grp – for administrative operations
- app_runtime_grp – for normal application operations

Read-only privileges were not assigned to a separate role because it was not included in the identified access requirements.

5.5 Default Privileges

- Default privileges were configured for future tables and sequences created by the postgres role in the public schema.
- The administrative group (app_admin_grp) receives ALL PRIVILEGES on future tables and sequences.
- The runtime group (app_runtime_grp) receives SELECT, INSERT, UPDATE, and DELETE privileges on future tables, and USAGE and SELECT privileges on future sequences.
- No default privileges were assigned to a separate read-only group because no read-only role was created.

6. Access Control Testing

The two login roles were tested separately to verify that their assigned privileges worked as expected.

The pawpaw_app account was tested for normal application operations, while the pawpaw_dba account was configured for administrative operations. Access restrictions were also checked to ensure that the runtime account could not perform unauthorized database and role creation operations.

6.1 Runtime Account

The following tests were performed:

Operation	Expected Result	Actual Result
SELECT	Allowed	Successful
INSERT	Allowed	Successful
UPDATE	Allowed	Successful
DELETE	Allowed	Successful

CREATE TABLE	Denied	Permission denied
CREATE DATABASE	Denied	Permission denied
CREATE ROLE	Denied	Permission denied

The SELECT operation successfully retrieved records from the pet table. INSERT and UPDATE operations were tested inside transactions and rolled back afterward.

The DELETE operation was also successfully performed using an unreferenced pet record. The transaction was rolled back afterward to restore the original record.

The runtime account was also unable to create tables, databases, or PostgreSQL roles. These results confirmed that the account can perform the required data operations but cannot perform administrative database structure or role creation tasks.

6.2 Read-Only Account

No separate read-only account or group role was created for the Pawpaw Clinic database.

The access-control configuration only includes two group roles:

- app_admin_grp – for administrative database operations
- app_runtime_grp – for normal application operations

Therefore, no read-only account testing was performed.

6.3 Administrative Account

The administrative account successfully connected to the group3_pawpaw database and performed the following tests:

Operation	Expected Result	Actual Result
SELECT	Allowed	Successful
INSERT	Allowed	Successful
UPDATE	Allowed	Successful

DELETE	Allowed	Successful
CREATE TABLE	Allowed	Successful
CREATE DATABASE	Denied	Permission denied
CREATE ROLE	Denied	Permission denied

The INSERT, UPDATE, and DELETE operations were tested inside transactions and rolled back afterward to avoid permanently changing the sample data.

The account also successfully created a test table inside the public schema. The test table was created within a transaction and rolled back afterward.

Although the account has administrative privileges on the existing database objects, it does not have the PostgreSQL CREATEDB, CREATEROLE, or SUPERUSER attributes.

These results confirmed that the administrative account can perform the tested database operations through its membership in app_admin_grp, while remaining restricted from creating databases and PostgreSQL roles.

6.4 Testing Summary

The tests confirmed the following:

- The pawpaw_dba account successfully inherited administrative privileges from app_admin_grp.
- The pawpaw_app account successfully inherited runtime privileges from app_runtime_grp.
- The runtime account can perform SELECT, INSERT, UPDATE, and DELETE operations.
- The runtime account cannot create tables, databases, or PostgreSQL roles.
- The administrative account can create tables but cannot create databases or PostgreSQL roles.
- Both login accounts have CONNECT permission on group3_pawpaw.
- The runtime account has USAGE permission but no CREATE permission on the public schema.
- The administrative account has both USAGE and CREATE permissions on the public schema.
- PUBLIC CONNECT access was revoked from the Pawpaw database.

6.5 Application Integration Testing

The Pawpaw Clinic application uses Node.js and PostgreSQL. The backend was tested using the dedicated pawpaw_app runtime account.

A direct database connection test confirmed that the application runtime account could successfully connect to PostgreSQL.

An API login test was also performed using invalid test credentials. The application returned the expected invalid username/email or password response. This confirmed that the application returned the expected authentication error instead of a database authentication failure.

Together, these tests helped verify that the Node.js backend could use the dedicated runtime account without requiring a superuser or database administrator account.

7. Issues and Limitations

During the implementation of the database roles and privileges, several setup considerations and limitations were encountered.

7.1 PostgreSQL Environment Setup

PostgreSQL 18.6 was used to configure the Pawpaw Clinic database.

The group3_pawpaw database was successfully initialized using the project's database scripts. Its existing tables, sequences, and sample records were inspected before assigning privileges.

The final database inspection confirmed that the public schema contained 14 tables and 12 sequences after the temporary testing objects were removed.

7.2 Role Configuration and Permissions

The group roles and login accounts were successfully created and configured.
One important consideration was that granting privileges to an administrative role does not automatically transfer ownership of existing database objects. The existing tables and sequences remained owned by postgres.

The pawpaw_dba account received the necessary privileges through its membership in app_admin_grp. However, ownership-dependent operations were not included in the administrative account tests.

Both login accounts were also verified to have the correct role memberships and restricted PostgreSQL administrative attributes.

7.3 Application Integration

The Pawpaw Clinic Node.js application was tested using the dedicated pawpaw_app runtime account.

The direct database connection and API authentication tests were completed successfully. The invalid-credentials test returned the expected application response.

The integration testing focused on verifying the database connection and runtime account access. It did not serve as a complete functional test of every application feature.

7.4 Default Privileges

Default privileges were configured specifically for future tables and sequences created by the postgres role in the public schema.

These settings apply to future objects created by that role. They do not automatically change the privileges of existing objects or objects created by other database owners.

No separate read-only group role was created because the identified database access requirements only included administrative and runtime roles.

Despite these considerations, the database role configuration, privilege restrictions, and planned verification tests were completed successfully.

8. Evidence and Repository

The SQL script and supporting screenshots for the Pawpaw access-control implementation are stored in the group's Car Rental Management System repository under db-access-control/Group03/.

Repository links:

- **Repository:** https://github.com/123airabels/Car_Rental_Management_System

- **Pawpaw access-control folder:** Car_Rental_Management_System/db-access-control/Group03 at main · 123airabels/Car_Rental_Management_System
- **SQL script:** Car_Rental_Management_System/db-access-control/Group03/pawpaw-roles-and-privileges.sql at main · 123airabels/Car_Rental_Management_System
- **Evidence folder:**
- Car_Rental_Management_System/db-access-control/Group03/evidence at main · 123airabels/Car_Rental_Management_System

8.1 Evidence Screenshots

The evidence screenshots cover the following configurations and tests:

1. Role configuration – group roles and login roles
2. Role memberships – login roles assigned to their intended group roles
3. Database privileges – CONNECT permission and PUBLIC access restriction
4. Schema privileges – USAGE and CREATE permissions
5. Table privileges – effective SELECT, INSERT, UPDATE, and DELETE permissions
6. Role security settings – LOGIN, SUPERUSER, CREATEDB, and CREATEROLE attributes
7. Runtime account tests – successful SELECT, INSERT, UPDATE, and DELETE operations
8. Runtime restrictions – denied CREATE TABLE, CREATE DATABASE, and CREATE ROLE operations
9. Administrative account tests – successful SELECT, INSERT, UPDATE, DELETE, and CREATE TABLE operations
10. Administrative restrictions – denied CREATE DATABASE and CREATE ROLE operations
11. Sequence privileges – effective sequence permissions for both login accounts
12. Final security verification – role memberships, database privileges, schema privileges, and object counts

8.2 GitHub Upload

The evidence screenshots were committed and pushed to the main branch.

The SQL script and additional security-test screenshots were committed and pushed successfully to the repository.

9. SQL Scripts Summary

The pawpaw-roles-and-privileges.sql script contains the SQL commands used to configure database roles, assign privileges, and apply access restrictions for the Pawpaw Clinic database.

The script covers the following operations:

- Group role creation
  Created app_admin_grp and app_runtime_grp as NOLOGIN roles.
- Login role creation
  Created pawpaw_dba and pawpaw_app as LOGIN roles.
- Role membership
  Assigned pawpaw_dba to app_admin_grp and pawpaw_app to app_runtime_grp.
- Database access
  Revoked CONNECT permission from PUBLIC on group3_pawpaw and granted CONNECT to the two intended group roles.
- Schema privileges
  Granted USAGE and CREATE permissions on the public schema to app_admin_grp.
  Granted USAGE only to app_runtime_grp.
- Administrative privileges
  Granted ALL PRIVILEGES on existing tables and sequences in the public schema to app_admin_grp.
- Runtime privileges
  Granted SELECT, INSERT, UPDATE, and DELETE on existing tables to app_runtime_grp. Also granted USAGE and SELECT on existing sequences.
- Default privileges
  Configured privileges for future tables and sequences created by the postgres role in the public schema. The admin group receives ALL PRIVILEGES, while the runtime group receives the permissions required for normal application operations.
- Access-control verification
  Included PostgreSQL queries to check role attributes, memberships, database and schema privileges, table privileges, sequence privileges, PUBLIC database access, and the final number of database objects.

The script follows the principle of least privilege by assigning permissions based on the responsibilities of each database account. Passwords are configured separately and are not included in the SQL script.

10. Verification Results

The database roles and privileges were checked using PostgreSQL queries and direct login tests.

10.1 Runtime Account Verification

Operation	Expected Result	Actual Result
SELECT records	Allowed	Successful
INSERT	Allowed	Successful; transaction rolled back
UPDATE	Allowed	Successful; transaction rolled back
DELETE	Allowed	Successful; transaction rolled back
CREATE TABLE	Denied	Permission denied
CREATE DATABASE	Denied	Permission denied
CREATE ROLE	Denied	Permission denied

Account: pawpaw_app
Evidence:
Car_Rental_Management_System/db-access-control/Group03/evidence/[06]Runtime_account_tests at main · 123airabels/Car_Rental_Management_System

10.2 Administrative Account Verification

Account: pawpaw_dba

Operation	Expected Result	Actual Result
Connect to database	Allowed	Successful
SELECT records	Allowed	Successful
INSERT	Allowed	Successful; transaction rolled back
UPDATE	Allowed	Successful; transaction rolled back
DELETE	Allowed	Successful; transaction rolled back
CREATE TABLE	Allowed	Successful; transaction rolled back
CREATE DATABASE	Denied	Permission denied

Evidence:
Car_Rental_Management_System/db-access-control/Group03/evidence/[08] Admin Account Tests at main · 123airabels/Car_Rental_Management_System

10.3 Additional Privilege Verification

The following configurations were also checked:

- Group roles have the NOLOGIN attribute.
- Login roles have the LOGIN attribute.
- pawpaw_dba belongs to app_admin_grp.
- pawpaw_app belongs to app_runtime_grp.
- Both login roles have CONNECT permission on group3_pawpaw.
- Both login roles have USAGE permission on the public schema.
- Only the administrative group has CREATE permission on the public schema.
- The runtime account has SELECT, INSERT, UPDATE, and DELETE privileges on existing tables.
- The administrative account has ALL PRIVILEGES on existing tables and sequences.
- The runtime account has USAGE and SELECT privileges on existing sequences.
- Neither login role has SUPERUSER, CREATEDB, or CREATEROLE attributes.
- PUBLIC CONNECT permission was revoked from the Pawpaw database.

Evidence:

- https://github.com/123airabels/Car_Rental_Management_System/blob/main/db-access-control/Group03/evidence/[01]role_config.png
- https://github.com/123airabels/Car_Rental_Management_System/blob/main/db-access-control/Group03/evidence/[02]Role_Memberships.png
- https://github.com/123airabels/Car_Rental_Management_System/tree/main/db-access-control/Group03/evidence/[03]Database_and_schema_privileges
- https://github.com/123airabels/Car_Rental_Management_System/tree/main/db-access-control/Group03/evidence/[04]Table_and_column_privileges
- https://github.com/123airabels/Car_Rental_Management_System/tree/main/db-access-control/Group03/evidence/[05]Role_security_settings
- https://github.com/123airabels/Car_Rental_Management_System/tree/main/db-access-control/Group03/evidence/[09]%20Sequence%20Privileges

10.4 Additional Security Verification

Additional tests were conducted to verify that the Pawpaw database accounts follow the assigned access restrictions.

Test Account	Operation	Expected Result	Actual Result
pawpaw_app	CREATE DATABASE	Denied	Permission denied
pawpaw_app	CREATE ROLE	Denied	Permission denied
pawpaw_app	CREATE TABLE	Denied	Permission denied
pawpaw_dba	CREATE DATABASE	Denied	Permission denied
pawpaw_dba	CREATE ROLE	Denied	Permission denied
pawpaw_dba	CREATE TABLE	Allowed	Successful

The tests confirmed that both login accounts were restricted from creating databases and PostgreSQL roles. However, the administrative account retained permission to create tables within the public schema, while the runtime account did not.

These results demonstrate the separation between database object privileges and PostgreSQL server-level administrative privileges.

11. Issues and Fixes

The following issues and considerations were encountered during the Pawpaw Clinic database access-control implementation.

Issue / Consideration	Action Taken / Resolution	Status
Existing database tables and sequences were owned by postgres.	Granted the required privileges to the appropriate group roles while retaining the original ownership.	Addressed
Runtime account needed normal data access without administrative permissions.	Granted SELECT, INSERT, UPDATE, and DELETE privileges while restricting schema CREATE permission.	Resolved
Runtime account needed to be prevented from creating databases and PostgreSQL roles.	Tested CREATE DATABASE and CREATE ROLE using the runtime account. Both operations were denied.	Resolved

Administrative account needed database object management permissions.	Granted USAGE and CREATE on the public schema and ALL PRIVILEGES on existing tables and sequences.	Resolved
Future database objects needed appropriate access permissions.	Configured default privileges for tables and sequences created by postgres in the public schema.	Configured
Temporary testing objects affected the database object count.	Removed the temporary test table and verified that the database returned to 14 tables and 12 sequences.	Resolved

11.1 Additional Verification Completed

The following additional checks were completed after the initial access-control configuration:

- Verified that pawpaw_app cannot create a new database.
- Verified that pawpaw_app cannot create a PostgreSQL role.
- Verified that pawpaw_app cannot create tables in the public schema.
- Verified that pawpaw_dba can create tables in the public schema.
- Verified that pawpaw_dba cannot create databases or PostgreSQL roles.
- Checked the role memberships and effective privileges of both login accounts.
- Confirmed that the temporary testing table was removed and the original database objects were retained.

12. Feedback to the Owning Group

The following observations and recommendations were identified during the Pawpaw Clinic database access-control implementation.

Area	Observation	Recommendation
Database setup	The database structure and sample records were successfully inspected before configuring access privileges.	Maintain clear database setup and initialization instructions.

Database privileges	Separate administrative and runtime group roles were created.	Continue assigning permissions based on each account's actual responsibilities.
Runtime account	The runtime account can perform normal CRUD operations but cannot create tables, databases, or PostgreSQL roles.	Keep the application connected through its dedicated runtime account instead of a superuser account.
Administrative account	The administrative account has privileges on database objects but does not have SUPERUSER, CREATEDB, or CREATEROLE attributes.	Maintain the separation between database object privileges and PostgreSQL server-level privileges.
Object ownership	Existing tables and sequences remain owned by postgres.	Document object ownership and consider it when performing ownership-dependent administrative operations.
Default privileges	Future object privileges were configured for objects created by postgres.	Verify default privileges whenever new database owners or schemas are introduced.
Documentation	The SQL script and verification results document the access-control setup.	Keep the SQL script, access-control plan, and screenshots updated whenever permissions change.

13. AI Usage Declaration

AI tools, particularly ChatGPT, were used as supplementary assistance during the completion of this database access-control activity.

The AI tool was used to:

- Understand PostgreSQL roles, privileges, and access-control concepts.
- Review and explain SQL commands used in configuring database permissions.
- Understand PostgreSQL errors encountered during implementation and testing.
- Help prepare verification queries for checking roles, memberships, and privileges.
- Organize and improve the documentation of the database access-control implementation.

All SQL commands were reviewed and executed using PostgreSQL 18.6. The actual database configuration, test results, and screenshots were used as the basis of the documentation.

The group remains responsible for the implementation, verification, and final output. AI assistance was used only as a supporting tool and did not replace the group's actual database testing and evaluation.

14. Conclusion

The database access-control implementation for Pawpaw Clinic was successfully configured and tested using PostgreSQL 18.6.

Two group roles, app_admin_grp and app_runtime_grp, were created and assigned to their respective login accounts, pawpaw_dba and pawpaw_app. The permissions were configured according to each account's responsibilities.

The runtime account was able to perform SELECT, INSERT, UPDATE, and DELETE operations but was restricted from creating tables, databases, and PostgreSQL roles. Meanwhile, the administrative account was able to manage the tested database objects and create tables but was also restricted from creating databases and PostgreSQL roles. Neither account was assigned SUPERUSER privileges.

The implementation was verified through role membership checks, privilege queries, and actual database operations. Default privileges were also configured for future tables and sequences created by postgres in the public schema. The Node.js application was also tested through a direct database connection test and an API authentication test using invalid credentials.

Overall, this activity demonstrated the importance of role-based access control, the principle of least privilege, and proper privilege management in securing a database. It also helped the group

understand how to configure, test, and document database permissions based on actual system requirements.